Modbus TCP with Portenta Machine Control & Opta™

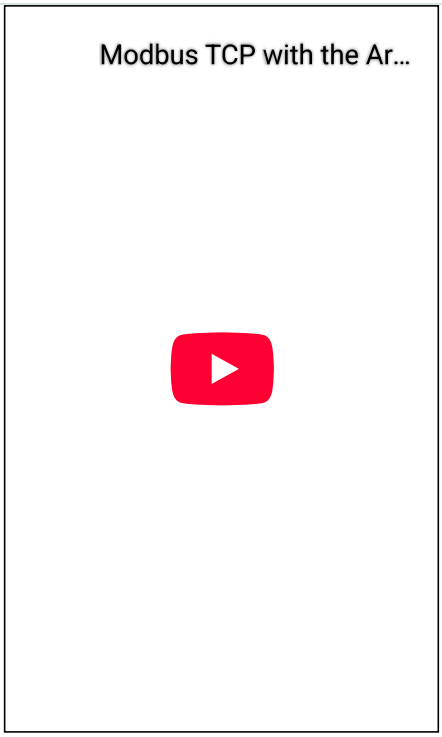
Learn to use Modbus TCP communication on a real industrial application using a Portenta Machine Control, Opta™, a temperature sensor, and the Arduino® PLC IDE.

Author · Christopher Mendez Last revision · 11/21/2024

Overview & Video Tutorial

In this tutorial, a Portenta Machine Control and an Opta™ micro PLC will be used as **server** and **client** respectively to share temperature information through Modbus TCP using the PLC IDE. The server will do the measurements using a type K thermocouple and the client will activate its relay outputs when a certain threshold is reached.

We have prepared a detailed guide in video format in case you are a visual learner.



If you prefer to follow the tutorial in a written format, or need to review the steps and code done in the video tutorial, in the following sections you will find a step-by-step guide explaining how the temperature measurement and Modbus communication between both devices were done.

Goals

- Learn how to measure temperature with the Portenta Machine Control using a thermocouple and the PLC IDE
- Discover how to use the Modbus protocol over TCP/IP using the PLC IDE
- Leverage Arduino Pro products for real industrial applications

Hardware and Software

Requirements

Hardware

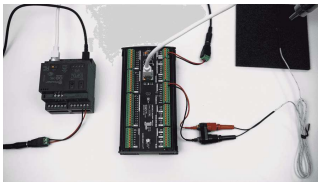
- Portenta Machine Control (x1)
- Opta™ (x1)
- Type K thermocouple (x1)
- Ethernet cables (x2)
- Wired internet access
- 24 VDC/0.5 A power supply (x2)

Software

- The Arduino PLC IDE
- Portenta Machine Control - PLC IDE Activation

Instructions

Solution Wiring



Real Application Wiring Setup

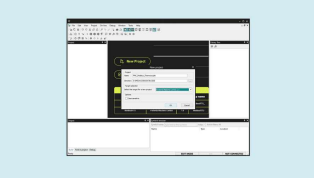
In the **Portenta Machine Control**: connect the thermocouple terminals to TP0 and TN0 respectively and the 24 VDC power supply to the 24-volt input and GND.

In the **Opta™ micro PLC**: connect the power supply to the respective inputs on the screw terminals.

Connect both the Portenta Machine Control and the Opta™ to your local network using ethernet cables.

Portenta Machine Control Setup

After downloading the PLC IDE, open it and create a **new project** for the Portenta Machine Control.



New project for the Portenta Machine Control

A license is needed for this product to be used with the PLC IDE, you can buy it directly from the Arduino store, and it will include the **product key** to activate the device.

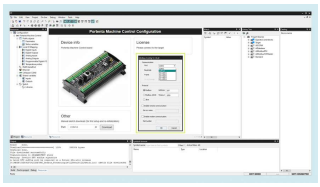
Connect the Machine Control to the computer using a micro USB cable, the board needs to run a specific program (runtime) in order to interact with the **PLC IDE**. To flash it, select the device serial port and click on download.

Runtime program download

Once the runtime is flashed, navigate to **On-line > Set up communication**,

select the **secondary** serial port, then click "OK".

Modbus properties

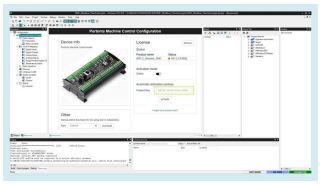


Modbus secondary serial port selection

Now, in the upper left corner, click on the **Connect** button and wait for the base program to be uploaded. A green **Connected** flag should appear in the lower right corner if everything goes well.

Connecting the board

The device will show its activation status, in this case, **No License** as it is the first time using it with the PLC IDE. To activate it, paste the **product key** you bought in the highlighted box and click on **Activate**.



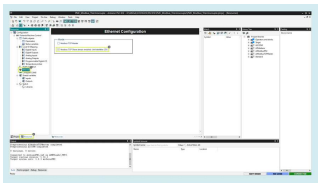
Activation process

programming the Portenta Machine Control with the PLC IDE.

To learn more about the PLC IDE first setup, continue reading this **detailed guide**.

Modbus TCP - Server

For the Modbus TCP configuration, on the **resources tab** go to the **Ethernet** section. As noticed, the Modbus TCP Slave mode is always enabled, so you don't have to make any changes.

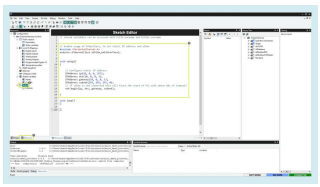


Modbus set to slave mode

Go to the **sketch editor** and uncomment the library and setup function code lines. As the IP, we must use the same as the Portenta Machine Control, as it is connected to your local network, you can find it on its configurations.

In this case, the following configurations are used:

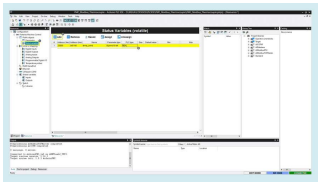
```
// Enable usage of EtherC
#include <PortentaEtherne
arduino::EthernetClass et

void setup()
{
    // Configure static I
    IPAddress ip(10, 0, 0
    IPAddress dns(10, 0,
    IPAddress gateway(10,
    IPAddress subnet(255,
    // If cable is not co
    eth.begin(ip, dns, ga

}
```



Network settings for Modbus TCP

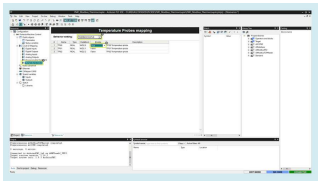
Now create the variable that will be shared with the temperature sensor data in the network. Go to **status variables** and click on **Add**. Configure it as follows:

- Name: `temp_send`
- Address: `25000`
- PLC type: `REAL`



Temperature variable setup

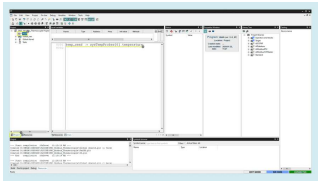
Next go to **Temperature probes** and select the sensor type, for this tutorial, enable the **thermocouple**



Temperature probe configuration

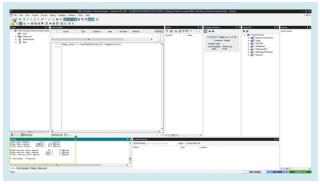
Finally, go to the **main program** in **project** and match the temperature variable with the temperature sensor lectures as follows:

```
1  temp_send := sysTempProbes
```



Main program for the server

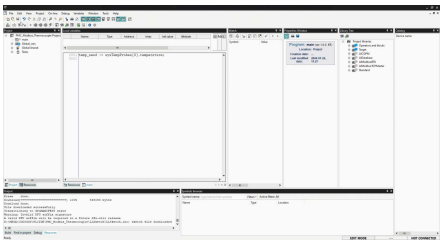
To check if everything is okay, click on compile, and if no error is shown, you can upload the code to your Portenta Machine Control.



Project compiled with no errors

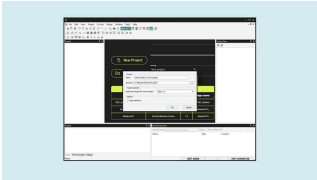
Once uploaded, click on the **Connect** button again. Now you can monitor the **temp_send** variable in the **Watch** window dragging and dropping it from the **Global shared** variables in

You should see the temperature value measured by the sensor inside the "Watch" window.



Opta™ Micro PLC Setup

Now the server is configured, create a new project, this time for the Opta™ micro PLC that will be the Client or Master.



New project for the Opta™

Upload the runtime for Opta™ by selecting its serial port and clicking on the **Download** button as before.

Uploading the runtime to the Opta™

Once the runtime is flashed, with your Opta™ connected to your router, search for its IP address on the router configurations.

On the PLC IDE, navigate to **On-line > Set up communication**, activate and then open the **ModbusTCP** properties. Add the Opta™ IP

Modbus TCP connection

Modbus TCP IP setup

Now, in the upper left corner, click on the **Connect** button and wait for the base program to be uploaded. A green **Connected** flag should appear in the lower right corner if everything goes well.

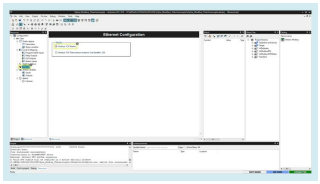
Connecting the board

The Opta™ is Pre-Licensed so you don't have to buy any license to use it with the PLC IDE

If the Opta status says **No License**, click on the **Activate PLC runtime** button to activate it. Learn more about this case in this guide.

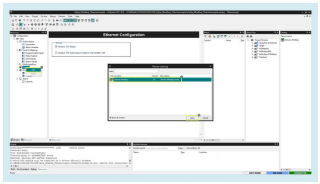
Modbus TCP - Client

With the Opta™ successfully connected to the PLC IDE, it is time to configure the Modbus TCP communication.



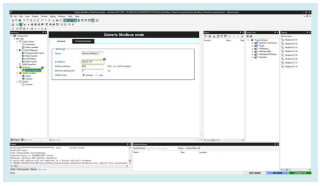
Modbus TCP Master mode enabled

Then right-click on the **Ethernet** tab, click on **Add** and select the *Generic Modbus device*.



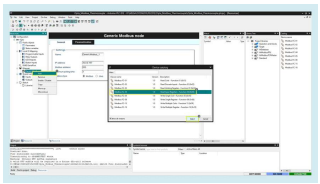
Modbus Device Configuration - Step 1

On the `Generic Modbus_1` device settings, enter the Server IP, the one from the Portenta Machine Control.



Modbus Device Configuration - Step 2

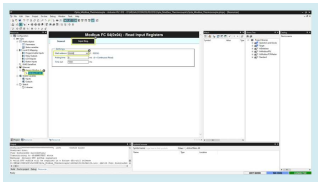
Right-click on the device and add the **FC-04 Modbus** function that will let us read the server input registers.



Modbus Device Configuration - Step 3

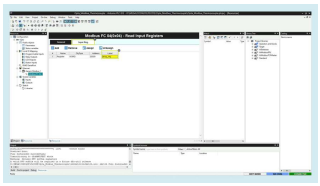
Now click on the function and in the

variable address that you defined earlier in the server, `25000` in this case.



Modbus Device Configuration - Step 4

In the Input Register tab, create a label for it, which could be `temp_reg`.

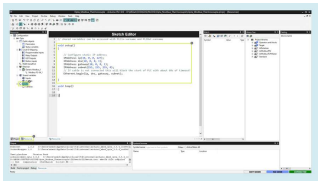


Label the register

Go to the **sketch editor** and uncomment the library and setup function code lines. As the IP, we must use the same as the Opta™.

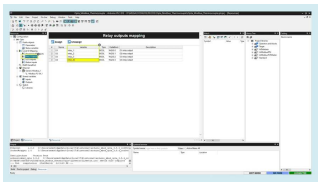
In this case the following configurations are used:

```
void setup()
{
    // Configure static I
    IPAddress ip(10, 0, 0
    IPAddress dns(10, 0,
    IPAddress gateway(10,
    IPAddress subnet(255,
    // If cable is not co
    Ethernet.begin(ip, dn
}
```



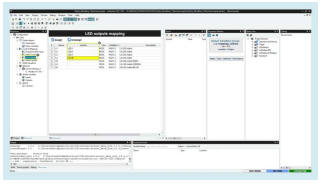
Network settings for Modbus TCP

Finally, define the Opta™ outputs behavior in function of the temperature read from the Portenta Machine Control. To do this, go to the **resources tab > Relay Outputs** and give a variable name to each relay, in this case, call them `relay_1`, `relay_2`, `relay_3` and `relay_4` respectively.



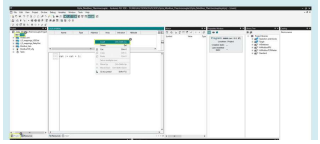
Outputs definitions

The same with the LED outputs, `LED1`, `LED2`, `LED3` and `LED4`.



LEDs definitions

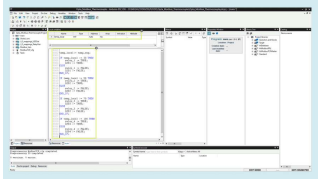
Now go to the main code in the Project tab. Create a variable by right-clicking on the local variables window and then **insert**, call it **temp_local** and *integer* as the type.



In the code editor, match the local variable with the shared one sent by the Portenta Machine Control.

Once stored locally, design the logic to control the outputs as you want. In this case, four different temperature levels will be set to control each output with the temperature rise respectively. Copy and paste the following script into the **main code** section.

```
temp_local:= temp_reg;

IF temp_local >= 30 THEN
    relay_1 := TRUE;
    LED1 := TRUE;
ELSE
    relay_1 := FALSE;
    LED1 := FALSE;
END_IF;

IF temp_local >= 50 THEN
    relay_2 := TRUE;
    LED2 := TRUE;
ELSE
    relay_2 := FALSE;
    LED2 := FALSE;
END_IF;

IF temp_local >= 70 THEN
    relay_3 := TRUE;
    LED3 := TRUE;
ELSE
    relay_3 := FALSE;
    LED3 := FALSE;
END_IF;

IF temp_local >= 100 THE
    relay_4 := TRUE;
```

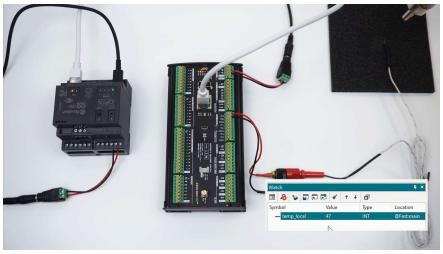


With the final code written, compile the project and upload it the the Opta™ micro PLC.

Final Test

You can leave each device connected separately to the internet router or connect them together directly with one ethernet cable. The first option will let you update the preferred device remotely as you can access it through the local network.

Now you can expose the temperature sensor to some heat and monitor it from the PLC IDE. The Opta™ relay outputs and LEDs will close and turn on when the temperature surpasses the programmed thresholds respectively.



Conclusion

In this tutorial you learned how to communicate two Arduino PRO products using the Modbus TCP protocol, demonstrating a simple application of sharing temperature data to control the outputs of the devices.

As you can notice, the configuration process is very straightforward and the results were as expected, being a good starting point to adapt the work done here to create your own professional solution.

Next Steps

Extend your knowledge about the Portenta Machine Control, PLC IDE and the variety of industrial protocols it supports by following these tutorials:

- Programming Introduction with Arduino PLC IDE
- Tank Thermoregulation with Portenta Machine Control & Opta™
- Connect an RTD/Thermocouple to the Portenta Machine Control
- Arduino PLC IDE Setup & Device License Activation

Suggest changes

The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here.

Need support?

Help Center
Ask the Arduino Forum
Discover Arduino Discord

License

Was this article helpful?